Admissibility:
What a Manipulation Benchmark Score Certifies

Anonymous submission

Abstract

A manipulation success rate can be made of *location* instead of *looking*. We make that precise. We define the *placement entropy* of a benchmark task — the entropy of its target's start pose over the states it ships — and prove the trivial but consequential fact that a suite with zero placement entropy is solved, at the demonstrator's own rate, by a lookup table indexed on the task. Measuring all four LIBERO suites, exactly one is admissible in this sense: LIBERO-Object carries 2.11 bits per task against a 5.64-bit ceiling, while Spatial, Goal and Long carry 5.50–5.62. We then make the proposition executable. On LIBERO-Object, a hand-written constant — one privileged read of the object's pose at reset, then no vision, no language and no learning — scores 10/10, *above* our own released policy at 8/10, while the same machinery given uninformed goals scores 0/10. The task is not trivial; the benchmark is. Run across the suite, that same released policy scores 8 of 40 — every success on the one object it was trained for, which is the admissibility argument turning on its author. We then audit a deliberately transparent 30M-parameter stack and locate where such a shortcut is read: four layers, two of them outside the model. Repairs are data-side and paired; one of our own is withdrawn here after powering it up. Finally, certification: probes alone cannot certify (two heads with attribution profiles agreeing to 0.73 cm score 0.000 and 0.700), and certificates are joint in *(head, stack)* — holding weights, seed and every package version fixed and changing only the renderer leaves the success *rate* identical and flips 40% of the individual episodes.

1 The question

A policy scores 0.700 on a manipulation benchmark. What has been certified?

The intended reading is a capability claim: the policy perceives the named object, locates it, and manipulates it. The reading this paper is about is narrower and much cheaper: the policy has learned *where the object always is*. Both readings produce the same number. Distinguishing them is not a matter of looking harder at the policy — two policies can be indistinguishable to a probe and opposite in the world, and we exhibit such a pair — but of first measuring the benchmark.

Recent perturbation studies have established the phenomenon at the level of scores. LIBERO-PRO [2] perturbs objects, initial states and instructions and reports models above 90% collapsing to 0.0%; LIBERO-Plus [1] perturbs seven dimensions across ten VLAs and concludes that models "tend to ignore language instructions completely." That VLAs ignore language is not our finding, and our system exhibiting it is confirmatory rather than novel.

What those results leave open is the part a practitioner needs. They show *that* scores collapse under perturbation. They do not say how much a given benchmark could have been gamed in principle, which stage of a stack is reading the shortcut, whether the shortcut can be removed, or what evidence would count as showing that it had been. This paper answers those four questions for one suite and one stack, in that order, and the first answer is the one that gives the others their meaning.

Contributions.

1. **Admissibility, defined and measured** (§2). Placement entropy H, a per-task quantity computed from the benchmark's shipped files alone — no policy, no simulator, no training. Measured for all four LIBERO suites, with per-task tables and file digests. LIBERO-Object is the outlier by a factor of 2.6 in bits.

2. **The proposition made executable** (§3). A two-constant memoriser that beats our released policy on the suite it was released for, bracketed by an oracle ceiling (10/10) and an uninformed floor (0/10) that together show the task is hard and the benchmark is not.

3. **A four-layer mechanism audit** (§6), including two layers outside the model — a scripted expert's calibration constant and our own checkpoint-selection loop — which black-box perturbation cannot reach.

4. **Certification methodology** (§8): a counterexample to probe-only certification, two annotation-free probes stated as protocols with their inconclusive conditions, and the demonstration that a behavioural certificate is joint in *(head, stack)* down to the renderer.

5. **Falsification record** (§10). Every prediction we registered, including the two that failed; every instrument retired at its calibration gate; and the repair claim of our own that this revision withdraws.

Scope, stated once and up front. All results are simulation. Every confirmatory cell is one task and one object unless stated. Cell sizes are given with every number and no cell is quoted without its interval. Our external validation attempt is reported as a *negative* (§9): we could not reproduce a public checkpoint's published performance on our build, and we do not report probe results on a harness whose baseline we cannot reproduce.

2 Admissibility

2.1 Definition

Fix a benchmark task t that ships a finite set S_t of initial states, and let $x_t(s) \in \mathbb{R}^3$ be the pose of the task's target object in state $s \in S_t$. Poses are compared at a physical tolerance δ, below which two placements are not distinguishable by the embodiment under test; we use $\delta = 1\,\text{mm}$ throughout, and report sensitivity to it.

Definition 1 (Placement entropy)**.** *Let q_δ quantise poses to a grid of side δ and let p_i be the empirical frequency of the i-th distinct value of $q_\delta(x_t(s))$ over $s \in S_t$. The* placement entropy *of task t is*

$$H_\delta(t) \;=\; -\sum_i p_i \log_2 p_i \quad \textit{bits},$$

and the placement entropy of a suite is the mean over its tasks.

$H_\delta(t) = 0$ says the target starts in the same place every time, to within δ. The ceiling is $\log_2 |S_t|$, attained when every shipped state places the target differently.

Two properties of the definition are worth stating because they bound what it can be used for. First, H_δ is *quantisation-dependent*: poses straddling a grid boundary can be split into two cells that a policy could not distinguish, so H_δ is an *upper* bound on the placement information a policy at tolerance δ can exploit. This direction is the safe one — it can overstate a suite's variation, never understate it, so a *low* measured H_δ is a conservative finding. We report H_δ across $\delta \in \{0.5, 1, 2, 5\}$ mm in Appendix A; the ordering of the four suites is unchanged throughout. Second, we report the mean over tasks rather than pooling states across tasks, because pooling would credit a suite for *between*-task variation — and between-task variation is exactly what a task-indexed lookup table gets for free.

Definition 2 (Order-k admissibility)**.** *A suite is* order-k admissible *if there is a map from the task index to a set of at most k pose constants that reproduces every shipped target pose to within δ.*

Proposition 1 (Lookup sufficiency)**.** *If $H_\delta(t) = 0$ for every task in a suite, then the suite is order-1 admissible, and a policy that (i) reads only the task index, (ii) emits the corresponding constant, and (iii) is otherwise driven by an identity-independent controller, achieves on that suite whatever success rate the controller achieves given a correct target pose.*

Proof. $H_\delta(t) = 0$ means the quantised pose $q_\delta(x_t(s))$ takes a single value on S_t; write c_t for any pose in that cell. For every $s \in S_t$, $\|x_t(s) - c_t\|_\infty \le \delta$, so the map $t \mapsto c_t$ reproduces every shipped target pose to within δ and the suite is order-1 admissible. The policy of the statement emits c_t on every state of task t, hence supplies the controller a pose within δ of the true one throughout. Since δ is by construction below the tolerance at which the embodiment distinguishes placements, the controller's trajectory — and therefore the episode outcome — is the one it would produce given the true pose. Averaging over S_t gives the claim. □

Corollary 1 (No score can separate them)**.** *On a suite with $H_\delta \equiv 0$, no function of episode outcomes alone can distinguish the lookup policy of Proposition 1 from a policy that localises the target perceptually and is otherwise identical, because the two induce the same distribution over trajectories.*

Corollary 1 is the sentence the rest of the paper exists to make concrete. It says the failure is not one of statistical power — no number of trials helps, because the two policies are not merely hard to tell apart but *identically distributed* in what the benchmark observes.

Proposition 1 is trivial as mathematics and is stated because its contrapositive is what benchmark scores are usually read as asserting. A suite with $H_\delta = 0$ cannot, by any score it reports, distinguish a policy that perceives from one that looks up — and the gap between them is exactly the capability the score is normally taken to certify. Note what the proposition does *not* say: it does not say the task is easy. The controller's own success rate under a correct pose is an empirical quantity, and if it is low the lookup policy scores low too. We measure both in §3.

2.2 Measurement

H_δ is computed from the benchmark's shipped files. No policy, no simulator and no training are involved, which is what makes it a property of the benchmark rather than of anyone's system.

LIBERO ships each task's initial states as a MuJoCo flattened state $[\,t \mid q \mid \dot{q}\,]$; every shipped $\dot{q}$ is zero, so the columns that vary across states are exactly the position degrees of freedom the suite randomises. Recovering *which* object owns which column requires care: the fixed layout $[\,t \mid 9 \mid N \times 7 \mid \dot{q}\,]$ that works for LIBERO-Object predicts a width of $19 + 13N$ and fails for every

suite	pinned	quat. id.	$H_{1\text{mm}}$	distinct
Object	**6/10**	10/10	**2.11**	**17.0**
Spatial	0/10	10/10	5.50	46.9
Goal	0/8	8/8	5.60	49.0
Long	0/10	9/10	5.62	49.3

Table 1: Placement entropy of every LIBERO suite, 50 shipped states per task, ceiling $\log_2 50 = 5.644$ bits. **pinned** counts targets taking ≤ 2 distinct `float64` positions; **quat. id.** counts tasks whose target start *orientation* is bit-identical across all 50 states. Denominators are *resolvable* tasks: two LIBERO-Goal tasks name a fixture with no free joint and have no placement to measure, so they are excluded and counted rather than scored as unpinned.

suite containing an articulated fixture (LIBERO-Spatial ships width 92 against 5 declared objects). We therefore compile each task's model and read `jnt_qposadr` with the joint names, which maps every column onto a named body. The two passes agree where both apply.

Table 1 is the result. Three readings.

One suite is admissible and three are not. LIBERO-Object carries 2.11 bits per task; Spatial, Goal and Long carry 5.50–5.62 against a 5.644 ceiling, i.e. 97–100% of the maximum. In 6 of Object's 10 tasks the target's start position takes exactly *two* distinct `float64` values, one unit in the last place apart ($\sim 3 \times 10^{-17}$ m) on a single axis — constant to the precision the format can express. The remaining four vary by 0.26–0.58 cm. Across tasks the structure is tighter still: the ten targets occupy two table positions 22.1 cm apart, five tasks each.

This is not carelessness, and saying so sharpens the finding. LIBERO-Spatial's ten tasks all name a "black bowl" and are individuated *by* position; freezing placement would make the suite unsolvable. LIBERO-Object individuates by identity and therefore *can* freeze placement without ambiguity. The suite that does is precisely the suite whose scores are read as evidence of object grounding — which is where an admissible benchmark is most misleading.

One property generalises. The target's start *orientation* is bit-identical across all 50 states in 37 of the 38 resolvable tasks in all four suites. A policy may assume a canonical target rotation suite-wide and never be contradicted.

3 The constant that beats the policy

Proposition 1 says a lookup table suffices on a zero-entropy suite. This section runs one.

The vehicle is our own stack (§5): a learned goal head that emits a world grasp point, driving a non-learned pick-and-place controller. That factorisation is what makes the experiment clean — we can replace the *goal supply* and change nothing else. Same controller, same trunk, same protocol, same seeds, same initial states; only where the goal comes from differs.

- **oracle** — the simulator's true target pose, re-read every tick. The controller's *ceiling*.
- **random** — a draw from the observed extent of the scene's own objects, re-drawn per episode. The controller's *floor*: what the machinery scores when the goal carries no information about where the object is.
- **constant** — one privileged read of the true pose at reset, held for the episode and never re-read. On a task whose placement is pinned this is numerically identical to a hardcoded constant, which is the point: it is Proposition 1's lookup policy, made executable.
- **learned head** — our released policy, for reference.

goal supply	score	Wilson 95%	vs. learned
oracle	10/10	$[0.72, 1.00]$	$p = 0.50$
constant	**10/10**	$[0.72, 1.00]$	$p = 0.50$
learned head	8/10	$[0.49, 0.94]$	—
random	**0/10**	$[0.00, 0.28]$	$\mathbf{p = 0.0078}$

Table 2: Goal-supply ablation on the held-out band, identical states and draws; only the goal supply differs. Comparisons against the learned head are paired per trial, exact McNemar. Per-trial outcomes: `results/pod_cells.json`.

The floor is zero. With an uninformed goal the machinery scores 0/10 $[0.00, 0.28]$: eight discordant pairs, all favouring the learned head, exact McNemar $p = 0.0078$. **The task is not trivial.** Whatever the constant achieves, it does not achieve it because the controller would have succeeded anyway. This is the control that makes the rest of the table mean something.

The constant matches the ceiling and beats the policy. The oracle scores 10/10, so the controller's envelope is not the binding constraint. The constant *also* scores 10/10 — one privileged look at reset, then no vision, no language, no learning — against the released head's 8/10. On a task whose target takes one `float64` value across all fifty shipped states, this is exactly what Proposition 1 predicts, and it is the difference between arguing that a benchmark admits memorisers and running one.

What this licenses, and what it does not. It does not show our released head is a memoriser; §6 shows what it actually reads, and the answer is more interesting. It shows that **on this suite the score cannot distinguish the two**, because the cheapest possible policy saturates it. A benchmark on which a constant is at ceiling is not measuring grounding, whatever its leaderboard is titled.

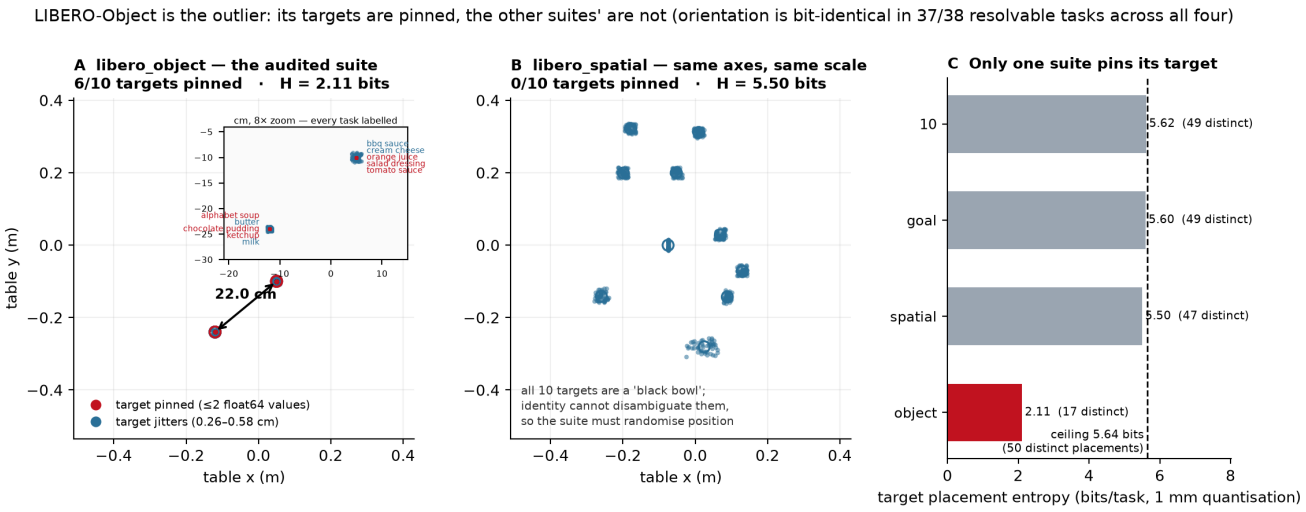

Figure 1: Placement forensics. **A** LIBERO-Object primary targets, 10 tasks × 50 shipped states; six collapse to a point and the ten task means occupy two clusters 22.1 cm apart. **B** LIBERO-Spatial on identical axes: every target scatters, and it must, because all ten of its targets are a "black bowl". **C** placement entropy per suite against the 5.644-bit ceiling. Generated by `paper/render_fig1_placement_forensics.py` from `results/suite_forensics_joints.json`; per-task rows and SHA-256 digests of every init-state array are in that artifact.

We report the head–oracle gap as *located* rather than measured: two discordant pairs at n=10 is $p = 0.50$, and we will not call that a difference.

One asymmetry, stated rather than buried. The anchors replace the *grasp* goal only; the place target is the true container in all arms. The random arm is therefore a *conservative* floor — an upper bound on how badly a truly uninformed goal supply would do — and we read it as such.

4 Recommendation to benchmark authors

Placement entropy is cheap to compute and cheap to report, and the measurement suggests three concrete practices. **Publish H_δ per task** with the suite; it costs a script and it tells a reader what a score can certify. **Set a floor.** A suite whose tasks are individuated by identity should still randomise placement, because otherwise identity is never load-bearing; LIBERO-Spatial does this by necessity and LIBERO-Object could do it by choice. **Ship a randomised tier.** Our own randomisation sweep (§7.2) shows that where a policy degrades depends on the displacement magnitude relative to the controller's envelope, so a single perturbation radius is a calibration, not a result — a tier of radii would let the community report a curve instead of a point.

5 A glass-box stack

The artifact is an instrument, not a contribution. It is built to be traceable end to end, and its size (30.2 M deployed) is what makes an end-to-end trace feasible rather than a claim about efficiency.

component	params	training	deployment
YOLO-World-S detector	13.0 M	never trained	frozen
world model (TRM)	9.97 M	stage A	frozen, inert[†]
trunk heads	7.0 M	stage A	frozen
goal heads + gates	0.24 M	stage B	frozen
deployed	**30.2 M**	17.2 M trained	all frozen

Table 3: Parameter ledger. There is *no* language model: the open-vocabulary detector's own text tower supplies every text embedding, so one frozen network is the only vision encoder *and* the only text encoder. [†]Inert in every cell we have ablated — a persistence baseline reproduces the held-out cell exactly. We previously wrote "inert in every nonzero number", which was a universal quantifier resting on one ablation.

Text is parsed into source/target phrases; the detector is prompted with them and returns a source box. A grasp head maps (uv, conf, proprio, box_emb, frame_emb) to a world grasp point — note the absence of any text argument, which is Figure 3's point and §8's architectural evidence. A place head maps the command embedding to a place (x, y), latched once per episode inside `reset()` and never re-read. A non-learned state machine converts goals into servo commands within a ±6 cm probe envelope.

Protocol, and exactly which states compose each cell. LIBERO at commit `8f1084e3`, MuJoCo 2.3.7, robosuite 1.4.1, torch 2.8.0, ultralytics 8.4.115, numpy 2.2.6, Python 3.10, `MUJOCO_GL=osmesa` (§8 explains why that last entry is not boilerplate). The harness sets `trial_seed` = `seed` · 1,000,003 + `trial` and indexes state `trial_seed` mod 50; since 1,000,003 ≡ 3 (mod 50), trial i uses state $(3\,\text{seed} + i) \bmod 50$. Every cell's state set is therefore derivable from its command line:

cell	seed, n	states
dev	0, 10	0–9
held-out	20, 10	10–19
held-out n=50	20, 50	**0–49**
untouched	40, 30	**20–49**

This answers a question we had not answered in print, and the answer is not flattering: **the n=50 cells are not disjoint from the dev and held-out bands — they contain them.** Fifty trials over fifty shipped states is the whole set. The untouched band exists to supply one clean cell, and §7 reports it.

6 The audit: four layers

"The policy memorised the location" is not one hypothesis but four, at four stages, two of which are not in the model at all — and those two are the ones a black-box perturbation study cannot reach (Figure 2).

6.1 Where the language goes

Swapping the instruction while holding the scene and the success predicate fixed collapses the released head from 8/10 to **0/20** $[0.00, 0.16]$; on the ten trials that pair with the reference cell, eight discordant pairs all one way, exact McNemar $p = 0.0078$. Telemetry places the failure *downstream of approach*: the policy still reaches the true object as closely as baseline while the grip-close rate falls from ∼0.67 to ∼0.26. Driving the detection prompts and the place-head embedding from *different* instructions isolates the collapse to one channel with no interaction. Combined with the architecture: **object selection, approach and grasping run with no language input at all, and the entire language channel is a single cached** (x, y). Ask this policy for the butter and it finds, approaches and grasps the alphabet soup at baseline rate; it fails only when that one coordinate is wrong.

7 Repairs, and one we withdraw

The untouched band. Both n=50 cells contain the burned states, so we ran the released head on states 20–49, which no selection look ever reached, with the prediction recorded before the run ($[0.50, 0.75]$, falsifiers named in both directions). Result: **20/30 = 0.667** $[0.49, 0.81]$, against the published 0.700 a difference of -0.033 with a Newcombe interval of $[-0.24, +0.16]$ on independent states. The five disclosed selection looks did not inflate the held-out figure. We claim only that: not that the burn is zero, but that it is small enough to hide inside ±0.20, which is what n=30 buys.

A repair we withdraw. We previously reported that command coverage repairs the instruction-swap behaviour, citing 3/10 against a 4/10 baseline at $p = 1.0$. A referee objected that this is an equivalence claim from an underpowered cell, and the objection was correct. Re-run on a band where the coverage head scores well, and taken to n=30:

head	baseline	swapped	paired McNemar
flagship	8/10	**0/20**	$p = 0.0078$
coverage	23/30	**13/30**	$p = 0.031$

At n=10 this contrast was $p = 0.125$ and we would have called it unresolved; at n=30 it is $p = 0.031$. **The coverage head collapses too.** A swap cannot visibly break a cell that is already mostly broken, which is why the original 4/10 baseline hid the effect.

What replaces the withdrawn claim is more useful than it was. Coverage does not eliminate the failure, it roughly *halves* it: the flagship loses its entire cell (0.800 → 0.000, every discordant pair one way), the coverage head about a third (0.767 → 0.433). And the repair to the place *point* is not in doubt — 14.15 → 0.78 cm, measured directly rather than inferred from behaviour. So: **command coverage fixes where the head aims and does not fully fix what it does.** A residual command→location dependence survives training on all three commands. That is a sharper statement of layer 4 than we had, and no cell we ran before could have detected it.

7.1 What the released policy actually is

The cells above are one task. Running the released head across the suite gives the number that puts them in proportion:

task	object	score	Wilson 95%
0	alphabet soup (trained)	**8/10**	$[0.49, 0.94]$
1	cream cheese	**0/10**	$[0.00, 0.28]$
2	salad dressing	**0/10**	$[0.00, 0.28]$
3	bbq sauce	**0/10**	$[0.00, 0.28]$

Eight successes in forty trials, and every one of them on the single object the head was trained for. We report this because it is the honest scale of the artifact and because it is the admissibility argument turning on its author: a suite of ten tasks that share two placements and one basket let a single-object solution be described as a policy with a suite-level score. The number a reader should carry away from this paper is not 0.700; it is 0.700 *on task 0*, with 0.000 next to it three times.

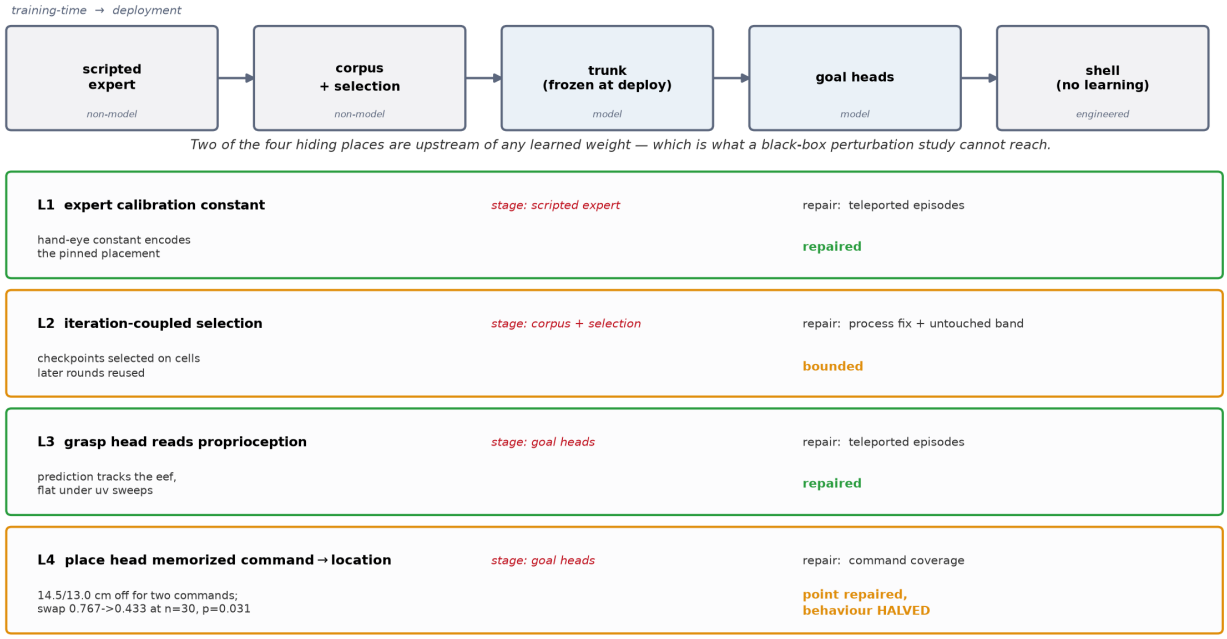

Figure 2: Where a fixed placement can hide in one stack, and the status of each site. Two of the four sit upstream of any learned weight.

This also bounds what §6's repairs bought. They made one object work and made the failures on the others legible — the drift case study (Table 4, last row) is one of those objects, lifted from 3/50 to 22/50 by DAgger on the policy's own viewpoints. They did not produce a general policy, and no cell in this paper should be read as claiming one.

7.2 Randomisation is a curve, not a point

Our ±4 cm radius was chosen to sit inside the controller's ±6 cm probe envelope — a perturbation calibrated to the system under test. Sweeping it on the same draws separates two things that single number confounded:

displacement	released head	Wilson 95%
none	4/10	[0.17, 0.69]
±2 cm	7/10	[0.40, 0.89]
±4 cm	4/10	[0.17, 0.69]
±6 cm (envelope edge)	2/10	[0.06, 0.51]
±8 cm (outside)	3/10	[0.11, 0.60]

Read conservatively, because n=10 supports no more: clearly worse at ±6–±8 than at ±2, but every interval is ∼0.5 wide and the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. That non-monotonicity is the honest measure of what this n resolves. **No single radius should be quoted as "passes randomisation"** — including the ±4 cm one we previously quoted that way.

8 Certification

If a benchmark score cannot certify grounding, what can? This section reports three results about the certification problem itself, two of them negative.

8.1 Probes alone cannot certify

A substitution probe asks which input channels a head reads: swap one channel between two recorded ticks and measure how far the predicted grasp offset moves. It is annotation-free, cheap, and on-manifold — and that last property is fatal on its own.

head	uv	proprio	box_emb	frame_emb
v2	0.75	2.85	6.04	6.91
v2.1	0.71	2.13	6.26	7.58
v5 (released)	0.97	1.93	3.43	6.57

Movement in cm of the predicted grasp *offset*. The offset, not the absolute point: the head emits $\text{eef}_{xy} + \Delta$, so substituting proprioception moves the absolute prediction by the entire arm displacement for *any* head, and profiling that quantity reports ∼24 cm of "proprioception attribution" even for a repaired one. We made exactly this error when regenerating the measurement, and record it because it is the difference between a probe that answers "does this channel change where the head thinks the object is" and one that measures the parameterisation.

v2 and v2.1 differ by at most 0.73 **cm on any channel and score** 0.000 **and** 0.700 **deployed.** A probe evaluated on teacher trajectories is an on-manifold instrument and cannot, alone, certify off-manifold behaviour.

layer	where	evidence	repair	verification / status
1. expert calibration constant	scripted expert (non-model)	hand-eye constant encodes the pinned placement	placement teleportation	0/10 → 7/10 dev, paired. **Repaired**
2. selection loop	our own procedure (non-model)	checkpoints chosen on cells later rounds reused	process fix; bands frozen	untouched band 20/30 vs 0.700: no detected inflation. **Bounded, not removed**
3. grasp-head proprioception	`GraspPointHead`	prediction tracks the eef, flat under uv sweeps	same teleported episodes	attribution flips to visual. **Repaired**
4. place-head command→location	`PlaceHead`	correct basket for the trained command, 14.5/13.0 cm off for the other two; swap 8/10 → **0/20**, p=0.0078	command coverage	place *point* 14.15 → 0.78 cm **repaired**; swap *behaviour* 0.767 → 0.433, p=0.031 — **halved, not fixed**
(separate) appearance drift	deployment viewpoints	NN-cosine gap on the policy's own trajectory	DAgger on own viewpoints	3/50 → 22/50 vs 3/50 control, $p < 10^{-4}$. **Repaired**

Table 4: Master audit table. Layers 1–4 are the placement-memorisation layers; the last row is a drift case study listed separately because it is *not* one of them — conflating the two is an error our own earlier abstract made, claiming "four layers, three repaired" while listing a repair that addresses none of the four. Two rows are deliberately not marked repaired.

Probe evidence must be paired with behavioural randomisation.

8.2 An instrument only ever shown firing is not yet an instrument

Our cross-instruction detection-agreement probe asks whether two objects' prompt chains select the *same box from the same frame*; a high same-box rate says the grounding stage carries no object identity. Every published number from it was a firing. Running it against every other object in the scene splits cleanly:

counterpart	n	same-box	median dist.
cream cheese, butter, choc. pudding	6	**0.00**	0.817–0.818
tomato sauce	4	1.00	0.00069
bbq sauce, salad dressing	5	0.80	0.0043
ketchup	5	0.60	0.0048

The instrument registers difference when difference exists. The distances are bimodal — collisions at ∼0.004, separations at ∼0.82, nothing between — and a sweep over $\tau \in \{0.005, 0.01, 0.02, 0.05\}$ leaves every verdict unchanged. The split is interpretable and sharpens the claim: the chains that collide are the cans and bottles, the ones that separate are the boxes. **Deployed binding is blind *within a shape class***, which is precisely the regime LIBERO-Object's groceries occupy.

Two limits. Only 4–6 of 60 frames had both chains detecting, so these are small-n demonstrations of discrimination, not calibrated rates. And the contrast we originally designated the positive control — the basket — is not evaluable at all: the deployed source-area filter rejects basket-sized boxes by construction, so it compares zero frames. Our first version of the script read same_rate = 0 off that empty comparison and printed "instrument discriminates". A verdict now requires $n \geq 3$.

8.3 Certificates are joint in (head, stack), down to the renderer

We rebuilt the deployment stack on fresh hardware, pinning every version in Table 3's caption and verifying both load-bearing checkpoints by digest. The reference cell reproduces: 8/10 [0.49, 0.94] against a recorded 7/10, with 0.700 inside the interval.

Reproducing the *rate* is not reproducing the *experiment*. Holding seed, trial, weights and every package version fixed and changing only the OpenGL backend MuJoCo renders through:

	`osmesa`	`egl`
success rate	8/10	8/10
per-trial outcomes	**4 of 10 disagree** (2 each way)	

The aggregate is identical and 40% of the episodes are not. Thread count, by contrast, changes nothing — two `osmesa` runs at 4 and 128 threads agree on every logged field. Software and GPU rasterisation do not produce identical pixels, the detector is not invariant to that difference, and in this stack the detector is the only encoder.

Two consequences. Matching a published rate is weak evidence of having reproduced an experiment: the marginal can be right while the sample is different. And *any paired statistic computed across a renderer change is*

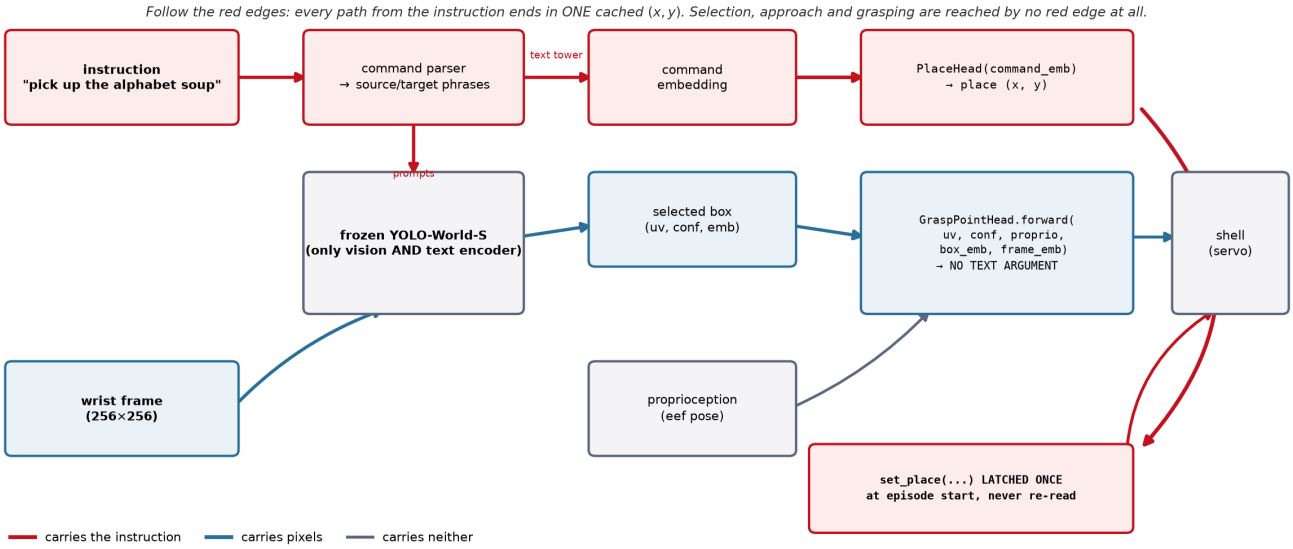

Figure 3: Every edge coloured by what it carries. The architectural half of the argument is a code fact: `GraspPointHead.forward` takes no text argument, so no red edge reaches object selection, approach or grasping, and every red edge terminates in one (x, y) latched once per episode and never re-read.

invalid, because pairing assumes the trials are the same trials — every McNemar in this paper is within one backend, which we can now say we checked rather than assumed. `MUJOCO_GL` belongs in any reported stack; it was not in ours, or in any LIBERO evaluation we have read.

9 Transfer: a negative result

The instruments above are offered as portable. We therefore attempted to run them on a policy we did not build: the public `openvla-7b-finetuned-libero-object` checkpoint, evaluated through the same harness, the same trial→state map and the same success predicate.

We report a negative, and the discipline that produced it matters more than the outcome. The baseline returned 0/1 on a checkpoint published near 0.9. That is a harness defect, not a finding about OpenVLA, and we treated it as one. Four defects were found and fixed: image orientation, the gripper convention (their model emits $[0, 1]$ with 1=close, robosuite wants $[-1, +1]$ with -1=close — without the conversion the jaw never closes and the policy *cannot* grasp), success extraction (the environment's `done` flag also fires on horizon), and a missing signal shield. We then swept four image orientations × the 0.9 centre crop their fine-tuned evaluation applies.

The checkpoint still does not approach the target: best end-effector distance 21.1 cm from a 31.5 cm start, where a working policy closes to under 5 cm. **We therefore report no OpenVLA numbers.** An unvalidated baseline would make our instruments look powerful for entirely the wrong reason — every one of the four defects biased in that same direction — and that is the easiest way to fabricate an external-validation result without intending to.

We cannot separate the two live hypotheses: a remaining defect in our harness, or a checkpoint that does not transfer to a differently-built LIBERO. The second is exactly what §8 predicts and we decline to claim it, because the first is not excluded. The harness ships (`eval/openvla_eval.py`) so the question is answerable by someone with the matching build. Until then, the portability of our instruments is **unestablished**, and we have removed the claim that "the audit generalises better than the artifact does" from this paper.

10 Negative results and registries

Pre-registration. Twelve predictions are on record with their outcomes. Five earlier ones, all falsified. Two from the confound test (§10.1). Seven registered for this revision, of which five held — the untouched band, the oracle ceiling, the uninformed floor, the constant, and the flagship swap — and two failed: that the coverage head's swap cell would hold (it does not, p=0.031) and that the released head would be at floor once displacement left the controller's envelope (3/10, not ≈ 0).

Retired instruments. Quantifying deployed role binding was attempted six times and abandoned six times, each at a calibration gate written down before the instrument was trusted: a projected-origin ground truth (rejected — the object that succeeds 35/50 scored 0.111), its sign-corrected successor (rejected — the in-frame filter deletes the close-up target by construction), three text-tower discrimination probes (rejected — each failed to detect the working object's own phrase), and a box-spread

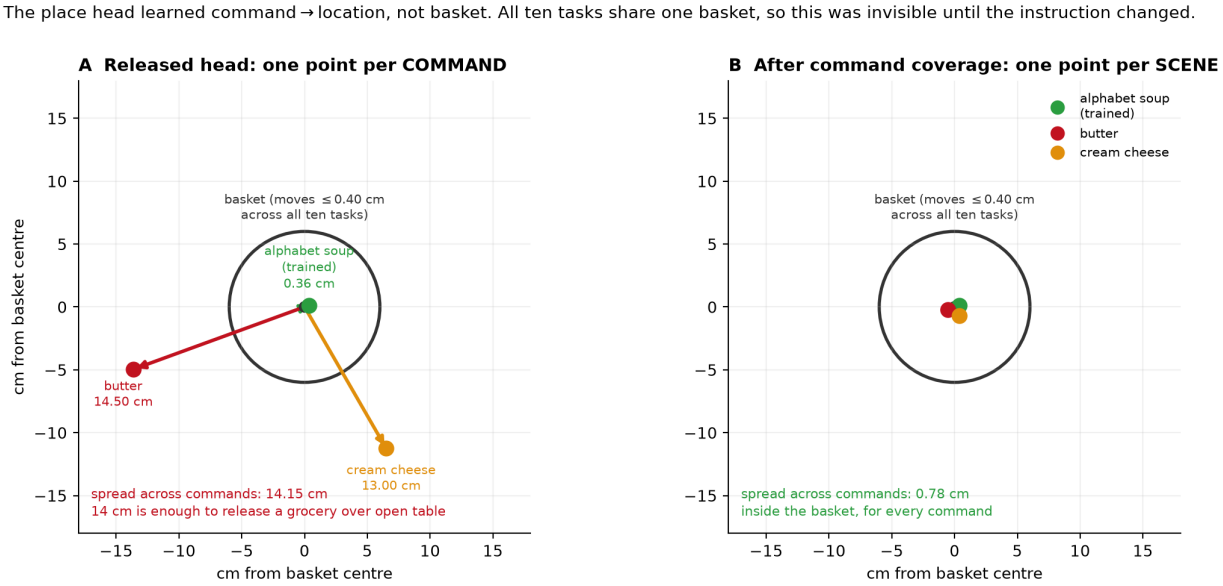

Figure 4: Layer 4 drawn rather than argued. The released head asked for three objects emits three place points, 14.5 and 13.0 cm from a basket that moves at most 0.40 cm across the entire suite; 14 cm is exactly enough to release a grocery over open table. Radial distances are measured; angles are for legibility.

metric (rejected — confounded by camera motion, and inverted). Deployed binding accuracy is reported as **unmeasured**.

10.1 A confound we had missed

Our own analysis of a blocked object was confounded: because the suite's two placement clusters align with which objects we happened to train on, "blocked" and "22 cm from the training position" were perfectly correlated in our design. Tested directly over all ten objects, position predicts the effect weakly (Spearman +0.19, p=0.60) and *teleporting each object 22 cm moves it in the wrong direction* (exact sign test p=1.0), while detector groundability predicts it (−0.71, p=0.027). The mechanism is appearance, not position — position is a nuisance term with mean $|\Delta| = 0.0066$, which is not zero either.

10.2 Three ways a result fooled us

Recorded because the failure modes generalise and none was caught by a conventional check.

A perfect null was a stale render. A teleport arm returned gaps identical to control for all ten objects to four decimals. The cause was a cached observation: the renderer returned the pre-teleport frame. The tell was implausible cleanliness. The fix is mechanical — the script now compares frames before and after the intervention and *raises* rather than emitting.

An unsourced retraction is an unsourced claim. We withdrew a defect count as unsupported after failing to find its appendix. The appendix existed. "I could not find it" is a claim about a search, not about a corpus.

A clean interim number was a censored sample. Harvesting a cell while its trials were still running gave 7/7 against a published 7/10, and we began writing up a failure to reproduce. The complete cell is 8/10. Successes end episodes early and failures run to the horizon, so the trials finished at any interim moment are biased toward successes. Every check we ran had passed; the defect was in *when* we looked. **Verifying the instrument does not verify the sampling.** The harvester now reports every cell against its planned n and refuses to report an incomplete one.

11 Discussion

What a score certifies. On a suite with $H_\delta \approx 0$, a success rate certifies that the policy can execute a manipulation given a correct target pose — and nothing about how it obtained one. That is not a rhetorical claim: on LIBERO-Object a constant achieves the ceiling and an uninformed goal achieves zero, so the score's entire dynamic range sits between "has the pose" and "does not", with grounding never tested. Placement entropy makes this checkable before any policy is trained.

Shortcuts live in pipelines, not only in models. Two of our four layers are outside the network: a scripted expert's calibration constant and our own checkpoint-selection loop. No amount of perturbing a trained model from the outside would have found either. This is an argument for glass-box artifacts in benchmark-validity work, and it is the main reason our stack is small.

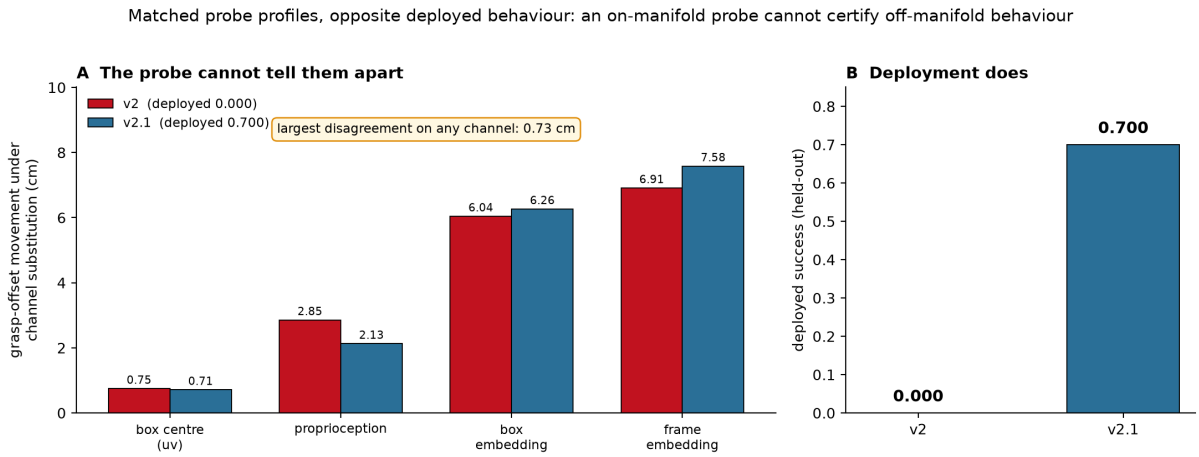

Figure 5: Two heads trained the same way on the same 10-episode corpus. Their attribution profiles agree to within 0.73 cm on every channel; deployed, they score 0.000 and 0.700. Regenerated by `scripts/attribution_profiles.py`.

Certification is a system property. Probes cannot certify alone (two heads, 0.73 cm apart, 0.000 vs 0.700). Behaviour must be paired with randomisation. And the certificate is joint in *(head, stack)* down to a component no requirements file pins: the same weights on the same seeds give the same rate and a different 40% of episodes under a different renderer.

Limitations. One simulator, one benchmark suite, one robot; wrist camera only; confirmatory cells are one task and one object; the held-out band is partially burned and the disclosure is in §5; the on-manifold limit of substitution probes is a limit on our own instruments; and transfer to an external policy is **unestablished** (§9). Placement entropy is defined for suites that ship enumerable initial states and would need restating for procedurally generated ones.

12 Conclusion

A manipulation benchmark score is not a measurement of capability until the benchmark's admissibility has been measured. We gave the measurement, showed that exactly one LIBERO suite fails it, and ran the constant that its failure predicts — which beats the policy we released for that suite. Where a real policy reads such a shortcut is then answerable, and we answered it at four layers, two outside the model. What survives is not the artifact but the procedure: measure the benchmark first, localise the stage second, repair with data third, and certify jointly in *(head, stack)* — reporting, as we have here, the repairs that did not survive being powered up.

A Sensitivity of H_δ to the tolerance

The quantisation δ is a modelling choice, so the finding must not depend on it. Recomputing placement entropy across four tolerances spanning an order of magnitude:

suite	0.5 mm	1 mm	2 mm	5 mm
Object	**2.21**	**2.11**	**1.77**	**1.28**
Spatial	5.57	5.50	5.29	4.62
Goal	5.64	5.60	5.45	4.77
Long	5.64	5.62	5.55	5.23

The ordering is unchanged throughout and the gap never falls below 2.5 bits. Coarsening δ lowers every suite's entropy, as it must — coarser bins merge distinct placements — and lowers LIBERO-Object's fastest in relative terms, because its variation is concentrated at submillimetre scales that a coarser grid erases entirely. Artifact: `results/entropy_delta_sensitivity.json`.

References

[1] Senyu Fei et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

[2] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.